ПРАКТИЧЕСКАЯ РАБОТА № 4
Базы данных, ORM Hibernate, Spring Data

ВВЕДЕНИЕ

В ходе выполнения практической работы предстоит ознакомится с основами база данных, а также изучить основные возможности фреймворков ORM Hibernate и Spring Data JPA.

Hibernate — фреймворк для языка программирования Java, предназначенный для решения задач объектно-реляционного отображения (ORM). Фреймворк является одной из самых популярных реализация спецификации JPA. Распространяется свободно на условиях GNU Lesser General Public License.

Hibernate позволяет сократить объемы низкоуровневого программирования при работе с реляционными базами данных. Используется как в процессе проектирования системы классов и таблиц «с нуля», так и для работы с уже существующей базой данных.

Фреймворк не только решает задачу связи классов Java с таблицами базы данных (и типов данных Java с типами данных SQL), но и также предоставляет средства для автоматической генерации, обновления набора таблиц, построения запросов и обработки полученных данных, и может значительно уменьшить время разработки, которое обычно тратится на ручное написание SQL- и JDBC-кода. Hibernate автоматизирует генерацию SQL-запросов и освобождает разработчика от ручной обработки результирующего набора данных и преобразования объектов, максимально облегчая перенос (портирование) приложения на любые базы данных SQL.

Hibernate обеспечивает использование SQL-подобного языка Hibernate Query Language (HQL), который позволяет выполнять SQL-подобные запросы, записанные рядом с объектами данных Hibernate. В Hibernate существует Criteria API, которая предоставляются как объектно-ориентированная альтернатива к языку HQL.

Наряду с Hibernate ключевой технологией для работы с БД в Java является Spring Data JPA. Spring Data - это часть экосистемы Spring, который упрощает разработку приложений на основе баз данных. Основная цель Spring Data заключается в абстрагировании деталей взаимодействия с различными типами хранилищ, будь то реляционные базы данных, NoSQL базы данных, графовые базы данных или другие. Spring Data предоставляет единый программный интерфейс и набор инструментов, позволяющих разработчикам сфокусироваться на бизнес-логике, снижая необходимость писать шаблонный код для операций с базой данных.

Одной из главных причин популярности Spring Data является его способность автоматически генерировать SQL-запросы на основе методов интерфейсов репозиториев. Это достигается путем анализа сигнатур методов и типов параметров, что позволяет разработчикам сосредоточиться на логике бизнеса, отпуская заботы о написании и оптимизации SQL-запросов.

Spring Data и Hibernate тесно связаны между собой, т.к. Spring Data использует концепцию JPA, которую, в свою очередь, реализуй фреймворк Hibernate. Но при этом стоит учесть, что Hibernate является не единственным провайдером JPA, поэтому Spring Data может работать и без Hibernate.

ОСНОВНЫЕ СВЕДЕНИЯ

В данном разделе представлены основные сведения об фреймворке Hibernate и Spring Data JPA необходимые для выполнения практической работы.

1. Основные положения

1.1. Entity (Сущность)

Entity (сущность) представляет собой класс Java, который связан с таблицей базы данных. Каждый объект сущности представляет запись в таблице. Для создания сущности в Hibernate, класс должен быть помечен аннотацией @Entity.

```
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
@Table(name = “users”)
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column
    private String name;

    // геттеры и сеттеры
}
```

Листинг 1 - пример класса-сущности User

Для более гибкого управления маппингом используется аннотация @Table, которая позволяет задать имя создаваемой таблицы и другие настройки. Для определения ключа в таблице используется аннотация @Id. Для включения автоматической генерации значений ключа используется аннотация @GeneratedValue. По аналогично с настройками таблицы используется аннотация @Column для настройки колонок таблицы.

1.2. Repository (Репозиторий)

Репозиторий – это интерфейс, который предоставляет часто используемые методы для выполнения операций над сущностями.

```
import org.springframework.data.repository.CrudRepository;
public interface UserRepository extends CrudRepository<User, Long>
{
}
```

Листинг 2 - пример репозитория для сущности User

В данном примере объявлен CRUD (create, read, update, delete) репозиторий. Существуют и другие более обширные реализации, такие как JpaRepository и PagingAndSortingRepository. После объявления репозитория, Spring, для сущности User, создаст не явную реализацию, которую можно будет внедрить в необходимые классы через объявленный интерфейс.

1.3. Query Methods (Методы запросов)

Query Methods – методы в репозитории, на основании имен которых будут сгенерированы запросы в БД.

```
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface UserRepository extends JpaRepository<User, Long> {
    List<User> findByName(String name);
}
```

Листинг 3 - пример именованного метода для поиска по полю

В данном примере на основании описанного метода Spring генерирует запрос для поиска сущности User по полю “Name”.

1.4. Аннотация @Query

Аннотация @Query позволяет создавать сложные запросы с использованием JPQL или SQL.

```
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface UserRepository extends JpaRepository<User, Long> {

    @Query("SELECT u FROM User u WHERE u.name = :name")
    List<User> findByName(@Param("name") String name);
}
```

Листинг 4 - пример использования аннотации @Query

Данный метод аналогичен методу из пункта 1.3. Аннотация @Query используется когда необходимо написать более сложные запросы, которые, например, невозможно описать с помощью Query Methods.

1.5. Criteria API

Criteria API в Java является частью JPA (Java Persistence API) и предоставляет типобезопасный способ создания запросов к базе данных. В отличие от JPQL, который использует строки для создания запросов, Criteria API использует методы и классы для построения запросов (т.е. объектный подход), что делает его удобным для динамического создания запросов и избежания ошибок на уровне компиляции.

```
// получаем билдер из Entity Manager (em)
CriteriaBuilder cb = em.getCriteriaBuilder();

// Создаем CriteriaQuery для сущности User
CriteriaQuery<User> cq = cb.createQuery(User.class);

// Определяем корневой элемент запроса, который соответствует User
Root<User> user = cq.from(User.class);

// Создаем параметр для имени
ParameterExpression<String> nameParam = cb.parameter(String.class);

// Создаем условие и добавляем его в запрос
cq.select(user).where(cb.equal(user.get("name"), nameParam));

// Создаем TypedQuery и подставляем значение параметра
TypedQuery<User> query = em.createQuery(cq);
query.setParameter(nameParam, "имя пользователя");

// Исполняем запрос и получаем результат
List<User> users = query.getResultList();
```

Листинг 5 - пример использования Criteria API

В данном примере представлен запрос аналогичный запросам из пунктов 1.3 и 1.4 только написанный с использованием Criteria API.

1.6. Связи между сущностями

В Spring Data JPA можно задавать различные виды связей (отношений) между сущностями.

1.6.1. One-to-One (Один-к-Одному)

Связь указывает на то, что каждая запись одной сущности соответствует только одной записи другой сущности. Для определения такой связи используется аннотация @OneToOne. Вместе с данной аннотацией используется аннотация @JoinColumn, которая определяет столбец являющийся внешним ключом в связанной таблице. Для примера, укажем связь между пользователем и его адресом:

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "address_id", referencedColumnName = "id")
    private Address address;

    // геттеры и сеттеры
}
```

Листинг 6 - пример связи один к одному для класса-сущности User

```
@Entity
public class Address {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String street;

    @OneToOne(mappedBy = "address")
    private User user;

    // геттеры и сеттеры
}
```

Листинг 7 - пример связи один к одному для класса-сущности Address

Данная структура означает что пользователь может иметь только 1 адрес, и адрес может принадлежать только 1-ому пользователю и никому больше.

1.6.2. One-to-Many (Один-ко-Многим)

Эта связь указывает на то, что одна запись одной сущности может соответствовать множеству записей другой сущности. Для определения такой связи используется аннотация @OneToMany. Например, один пользователь может иметь несколько заказов:

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
    private List<Order> orders;

    // геттеры и сеттеры
}
```

Листинг 8 - пример связи один ко многим для класса-сущности User

```
@Entity
public class Order {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String product;

    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    // геттеры и сеттеры
}
```

Листинг 9 - пример связи один ко многим для класса-сущности Order

Данная связь означает что пользователь может иметь несколько заказов, но заказ оформить может только 1 пользователь.

1.6.3. Many-to-One (Многие-ко-Одному)

Эта связь является обратной к связи "One-to-Many". На нашем примере это может означать что заказ могут оформить несколько пользователей. Для определения такой связи используется аннотация @ManyToOne.

1.6.4. Many-to-Many (Многие-ко-Многим)

Эта связь указывает, что каждая запись одной сущности может соответствовать множеству записей другой сущности, и наоборот. Для определения такой связи используется аннотация @ManyToMany. Например, отношения между студентами и курсами:

```
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany
    @JoinTable(
      name = "student_course",
      joinColumns = @JoinColumn(name = "student_id"),
      inverseJoinColumns = @JoinColumn(name = "course_id"))
    private Set<Course> courses;

    // геттеры и сеттеры
}
```

Листинг 10 - пример связи многие ко многим для класса-сущности Student

```
@Entity
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @ManyToMany(mappedBy = "courses")
    private Set<Student> students;

    // геттеры и сеттеры
}
```

Листинг 11 - пример связи многие ко многим для класса-сущности Course

В данном случае студенты могут посещать несколько курсов, а на курс может приходить множество студентов.

При использовании данной аннотации Spring создаст промежуточную таблицу, где будет определено однозначное соответствие между студентом и курсом. Такая реализация удобна небольшим кол-вом кода, однако, рекомендуется вручную создавать промежуточную таблицу и приводить все к связи “Один-ко-Многим” или “Многие-ко-Одному”, чтобы в дальнейшем избежать неявных проблем, скрытых реализацией Spring.

1.7. Транзакции

Транзакция — это логическая единица работы, которая состоит из одного или нескольких операций чтения и/или записи данных в базу данных. Основной целью транзакции является обеспечение атомарности, последовательности, изоляции и долговечности (ACID-свойства).

ACID-свойства транзакции:

- Атомарность (Atomicity). Все операции внутри транзакции либо выполняются полностью, либо не выполняются вовсе. Если в процессе выполнения возникает ошибка — все изменения откатываются;
- Последовательность (Consistency). Выполнение транзакции переводит систему из одного согласованного состояния в другое;
- Изоляция (Isolation). Результаты изменений, сделанных в рамках одной транзакции, не видны другим параллельно выполняющимся транзакциям до завершения текущей транзакции;
- Долговечность (Durability). Изменения, внесённые успешно завершенной транзакцией, сохраняются и не теряются даже в случае сбоя системы.

Далее рассмотрим несколько подходов для управления транзакциями.

1.7.1. Аннотация @Transactionl

Аннотация @Transactional в Spring Framework используется для обеспечения поддержки транзакций на уровне метода или класса. Она позволяет управлять транзакциями, не вмешиваясь в их явное открытие, подтверждение или откат, что делает код более чистым и читаемым. Транзакции гарантируют, что несколько операций над базой данных будут выполнены как одна единица работы: либо все операции завершатся успешно, либо ни одна из них.

```
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class UserService {

    private final UserRepository userRepository;
    private final RoleRepository roleRepository;

      public UserService(UserRepository userRepository, RoleRepository
roleRepository) {
        this.userRepository = userRepository;
        this.roleRepository = roleRepository;
    }

    @Transactional
    public void createUserWithRoles(User user, Set<Role> roles) {
        user.setRoles(roles);
        for (Role role : roles) {
            roleRepository.save(role);
        }
        userRepository.save(user);
    }
}
```

Листинг 12 - пример использования аннотации @Transactional

В этом примере метод createUserWithRoles помечен аннотацией @Transactional, что означает, что весь метод будет выполнен в одной транзакции. Если какое-то из сохранений (например, roleRepository.save(role) или userRepository.save(user)) не удастся, вся транзакция будет отменена.

1.7.2. PlatformTransactionManager

PlatformTransactionManager — это интерфейс в Spring, отвечающий за управление транзакциями. Он предоставляет абстракцию над основными транзакционными системами, такими как JDBC, JPA, JTA и Hibernate. Это позволяет разработчикам управлять транзакциями единообразным образом, независимо от используемой системы баз данных. Пример использования приведен в приложении Д.

1.8. Тестирование

Тестирование в Spring представляет собой комплекс подходов и инструментов, позволяющих разрабатывать и проверять приложения на платформе Spring Framework. Spring предоставляет обширную поддержку для юнит-тестирования, интеграционного тестирования и тестирования веб-слоев. Одним из ключевых компонентов в Spring является TestContext Framework, который упрощает управление контекстом приложения для тестов. На ряду с этим Spring поддерживает популярные фреймворки для тестирования, таких как JUnit и TestNG. С помощью аннотаций, таких как @SpringBootTest, @WebMvcTest, и @DataJpaTest, разработчики могут легко конфигурировать и запускать тестовые сценарии, максимально приближенные к реальным условиям работы приложения. Пример использования приведен в приложении Е.

ХОД РАБОТЫ

В данном разделе описан ход выполнения практической работы. В рамках работы предстоит реализовать базу данных и методы работы с ней в соответствии с темой вашего проекта.

1. Установка СУБД

В рамках работы предлагается использовать **СУБД Postgresql** (по желанию можно использовать любую другую СУБД). Процесс установки Postgresql описан на официальном сайте. Рекомендуется использовать последнюю стабильную версию.

После установки СУБД необходимо создать базу данных для последующего ее использования. Дальнейший ход работы будет подразумевать использование именно Postgresql.

2. Настройка проекта в IDE

В качестве проекта (git репозитория) необходимо использовать проект из предыдущего заданий (рекомендуется отключить консольный ввод из предыдущего задания).

Для того, чтобы использовать возможности Spring Data необходимо подключить зависимость в pom.xml:

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Листинг 13 - подключение зависимости Spring Data JPA

Для того, чтобы использовать Postgresql необходимо подключить зависимость:

```
<dependency>
   <groupId>org.postgresql</groupId>
   <artifactId>postgresql</artifactId>
   <scope>runtime</scope>
</dependency>
```

Листинг 14 - подключение зависимости Postgresql

Далее необходимо настроить подключение к СУБД из приложения в application.properties:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/<your_db>
spring.datasource.username=<your_login>
spring.datasource.password=<your_password>
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreS
QLDialect
spring.jpa.hibernate.ddl-auto=update
```

Листинг 15 - пример настроек application.properties для подключения к СУБД Postgresql

3. Проектирование структуры БД

Необходимо спроектировать структуру БД в зависимости от выбранной темы вашего проекта. Необходимо выбрать не менее 5 сущностей вашей предметной области. Для каждой из сущностей необходимо определить связи с другими сущностями и атрибуты (поля). Каждая сущность должна иметь хотя бы 1-у связь с другой сущностью, и в каждой сущности должно быть не менее 3-х логических атрибутов (не учитываются первичные и внешние ключи).

Для наглядного представления структуры БД используется ER-диаграммы. В качестве инструмента построения ER-диаграммы можно использовать любой доступный инструмент построения визуальных схем, например dbdiagram. Пример ER-диаграммы приведен в приложении А.

В результате должна быть реализована ER-диаграмма описывающая структуру БД вашего проекта.

4. Реализация структуры БД

Далее необходимо реализовать спроектированную структуру БД в коде используя возможности Spring Data. Для каждой сущности необходимо реализовать свой Java-класс (POJO) с соответствующими полями. Пример реализации класса-сущности приведет в приложении Б.

После реализации сущностей и запуска приложения структура БД должна соответствовать ER-диаграмме. Все сущности должны находится в отдельном пакете.

5. Реализация репозиториев для работы с данными

Для каждой сущности необходимо реализовать CRUD репозиторий на основании базового репозитория Spring *(org.springframework.data.repository.CrudRepository)*.

Далее необходимо в одном из репозиториев (на свое усмотрение) реализовать 1 метод поиска сущности по его полям с использованием наименования методов Query Lookup Strategies. Важным условием является использование в имени метода ключевых слов (And, Or, Between…).

Также необходимо в одном из репозиториев (на свое усмотрение) реализовать 1 метод поиска сущности через связанную сущность. Необходимо написать запрос на языке JPQL с использованием аннотации @Query.

Пример реализации CRUD-репозитория для сущности приведен в приложении В. Все репозитории должны лежать в отдельном пакете.

6. Реализация запросов с использованием Criteria API

Иногда возможностей Spring Data может быть не достаточно или его использование может быть не удобным. Поэтому, в больших проектах можно встретить использование Criteria API.

Необходимо реализовать кастомные методы (2 метода) из пункта 5 текущего раздела, только уже с использованием Criteria API. Для этого нужно создать отдельный интерфейс и реализовать его в отдельном классе.

Пример реализации запросов с использованием Criteria API приведен в приложении Г. Интерфейс и класс должны находится в отдельном пакете.

7. Реализация транзакционной операции

В зависимости от структуры вашей БД необходимо выбрать 2-е такие сущности, одна их которых, логически не может существовать без другой. Для этих сущностей необходимо реализовать транзакционный метод сервиса, который бы обеспечивал целостность данных при операции с данными сущностями (операцию можно выбрать на свое усмотрение: это может быть операция создания, обновления, изменения и т.д.). Метод должен быть реализован в отдельном сервисе. Транзакционность должна обеспечиваться с использованием механизма Transaction Manager или любого другого способа управления транзакциями предоставляемые Spring и Hibernate. Пример реализации транзакционной операции приведен в приложении Д.

8. Тестирование

Необходимо протестировать реализованные методы для работы с сущностями.

Необходимо написать автотесты для проверки методов реализованных в пункте 5 текущего раздела и аналогичных методов реализованных в пункте 6 текущего раздела.

Также необходимо написать автотесты для проверки метода с транзакционной операцией из пункта 7 текущего раздела. Тесты должны

покрывать как и положительный, так и отрицательный кейс с откатом транзакции.

Примеры автотестов приведены в приложении Е.

СДАЧА РАБОТЫ

Практическая работа считается успешно выполненной если выполнены все пункты из раздела “Ход работы”.

Критерии выполнения работы:

- спроектирована ER-диаграмма;
- реализована структура БД в коде;
- для каждой из сущностей реализованы репозитории;
- реализованы 2 метода в репозиториях с использованием именованных запросов и HQL, и реализованы аналогичные запросы с использованием Criteria API.
- реализован сервис с транзакционной операцией;
- написаны автотесты на кастомные методы репозитория и на транзакционный метод сервиса;
- создан pull request с изменениями по заданию.

После успешной сдачи работы необходимо все изменения по работе перенести в основную ветку приложения.

ПОЛЕЗНЫЕ ССЫЛКИ

1. Сайт Spring Framework
2. Документация Spring Framework
3. Spring Data
4. Hibernate
5. Документация HQL
6. Testing in Spring

ПРИЛОЖЕНИЕ А

Пример ER-диаграммы БД

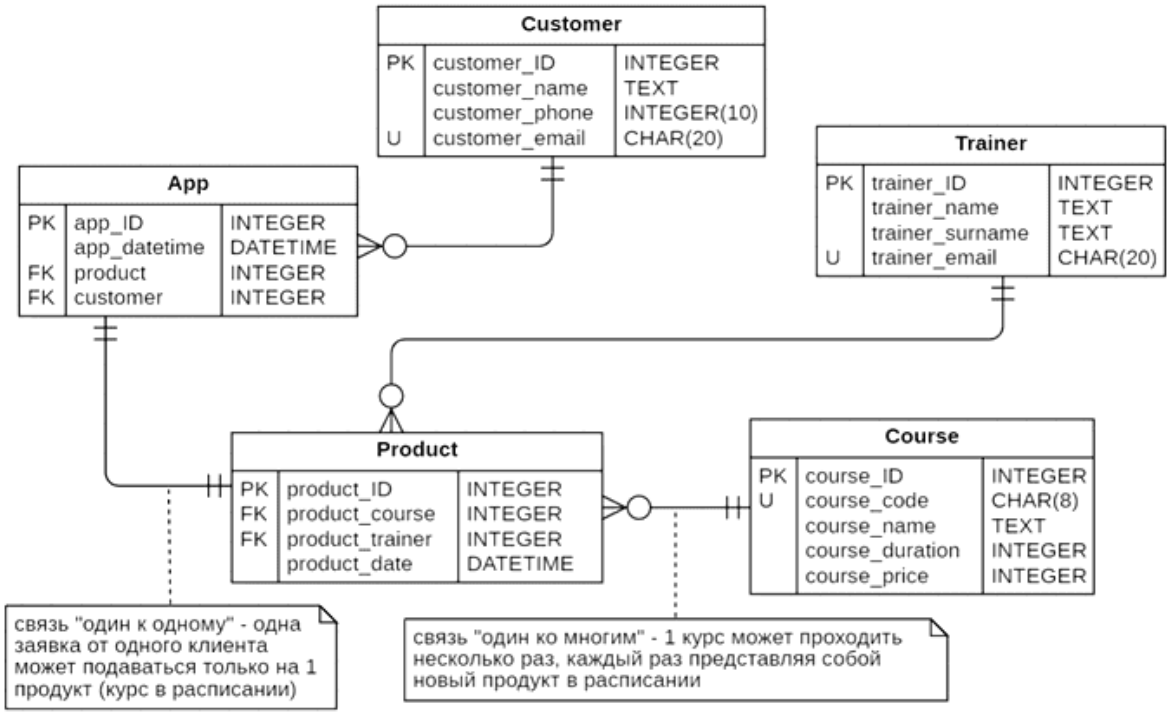


Рисунок 1 - пример ER-диаграммы

ПРИЛОЖЕНИЕ Б

Примеры класса-сущности

```
package ru.naumen.NauJava.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;

@Entity
@Table(name = "users")
public class User
{
    @Id
    @GeneratedValue
    private Long id;

    @Column
    private String name;

    @Column
    private String login;

    @ManyToOne
    private Role role;

    public Long getId()
    {
        return id;
    }

    public void setId(Long id)
    {
        this.id = id;
    }

    public String getLogin()
    {
        return login;
    }

    public void setLogin(String login)
    {
        this.login = login;
    }

    public String getName()
    {
        return name;
    }

    public void setName(String name)
    {
        this.name = name;
    }

    public Role getRole()
    {
        return role;
    }

    public void setRole(Role role)
    {
        this.role = role;
    }
}
```

Листинг 16 - пример класса-сущности User

```java
package ru.naumen.NauJava.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;

@Entity
public class Role
{
   @Id
   @GeneratedValue
   private Long id;

   @Column(unique = true)
   private String title;

   public Long getId()
   {
       return id;
   }

   public void setId(Long id)
   {
       this.id = id;
   }

   public String getTitle()
   {
       return title;
   }

   public void setTitle(String title)
   {
       this.title = title;
   }
}
```

Листинг 17 - пример класса-сущности Role

ПРИЛОЖЕНИЕ В

Пример объявления репозитория

```
package ru.naumen.NauJava.repository;

import java.util.List;

import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.CrudRepository;

import ru.naumen.NauJava.entity.User;

public interface UserRepository extends CrudRepository<User, Long>
{
   /**
    * Находит всех пользователей с заданным именем
    * @param name имя пользователя
    */
   List<User> findByName(String name);

   /**
    * Находит всех пользователей с заданным название роли
    * @param roleTitle наименование роли
    */
   @Query("FROM User WHERE role.title = :roleTitle")
   List<User> findByRole(String roleTitle);
}
```

Листинг 18 - пример объявления CRUD репозитория для сущности User

ПРИЛОЖЕНИЕ Г
Пример использования Criteria API

```
package ru.naumen.NauJava.dao;

import java.util.List;
import ru.naumen.NauJava.entity.User;

public interface UserRepositoryCustom
{
   /**
    * Находит всех пользователей с заданным именем
    * @param name имя пользователя
    */
   List<User> findByName(String name);

   /**
    * Находит всех пользователей с заданным название роли
    * @param roleTitle наименование роли
    */
   List<User> findByRole(String roleTitle);
}
```

Листинг 19 - пример объявления кастомного репозитория для сущности User

```java
package ru.naumen.NauJava.dao;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;

import jakarta.persistence.EntityManager;
import jakarta.persistence.criteria.CriteriaBuilder;
import jakarta.persistence.criteria.CriteriaQuery;
import jakarta.persistence.criteria.Join;
import jakarta.persistence.criteria.JoinType;
import jakarta.persistence.criteria.Predicate;
import jakarta.persistence.criteria.Root;
import ru.naumen.NauJava.dao.UserRepositoryCustom;
import ru.naumen.NauJava.entity.Role;
import ru.naumen.NauJava.entity.User;

@Repository
public class UserRepositoryImpl implements UserRepositoryCustom
{
   private final EntityManager entityManager;

   @Autowired
   public UserRepositoryImpl(EntityManager entityManager)
   {
       this.entityManager = entityManager;
   }

   @Override
   public List<User> findByName(String name)
   {
       CriteriaBuilder criteriaBuilder = entityManager.getCriteriaBuilder();
       CriteriaQuery<User> criteriaQuery = criteriaBuilder.createQuery(User.class);

       Root<User> userRoot = criteriaQuery.from(User.class);
       Predicate namePredicate = criteriaBuilder.equal(userRoot.get("name"), name);

       criteriaQuery.select(userRoot).where(namePredicate);

       return entityManager.createQuery(criteriaQuery).getResultList();
   }

   @Override
   public List<User> findByRole(String roleTitle)
   {
       CriteriaBuilder criteriaBuilder = entityManager.getCriteriaBuilder();
       CriteriaQuery<User> criteriaQuery = criteriaBuilder.createQuery(User.class);

       Root<User> userRoot = criteriaQuery.from(User.class);
       Join<User, Role> role = userRoot.join("role", JoinType.INNER);
       Predicate namePredicate = criteriaBuilder.equal(role.get("title"), roleTitle);

       criteriaQuery.select(userRoot).where(namePredicate);

       return entityManager.createQuery(criteriaQuery).getResultList();
   }
}
```

Листинг 20 - пример реализации репозитория для сущности User с использованием Criteria API

ПРИЛОЖЕНИЕ Д
Пример транзакционной операции

```
package ru.naumen.NauJava.service;

public interface RoleService
{
   /**
    * Удаляет роль по имени
    * @param roleTitle имя роли
    */
   void deleteRole(String roleTitle);
}

package ru.naumen.NauJava.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.dao.DataAccessException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.TransactionStatus;
import org.springframework.transaction.support.DefaultTransactionDefinition;

import ru.naumen.NauJava.entity.User;
import ru.naumen.NauJava.repository.RoleRepository;
import ru.naumen.NauJava.repository.UserRepository;

@Service
public class RoleServiceImpl implements RoleService
{
   private final RoleRepository roleRepository;
   private final UserRepository userRepository;
   private final PlatformTransactionManager transactionManager;

   @Autowired
   public RoleServiceImpl(RoleRepository roleRepository, UserRepository userRepository,
           PlatformTransactionManager transactionManager)
   {
       this.roleRepository = roleRepository;
       this.userRepository = userRepository;
       this.transactionManager = transactionManager;
   }

   @Override
   public void deleteRole(String roleTitle)
   {
       TransactionStatus status = transactionManager.getTransaction(new
DefaultTransactionDefinition());
       try
       {
           // удалить пользователей с данной ролью
           List<User> users = userRepository.findByRole(roleTitle);
           for (User user : users)
           {
               userRepository.delete(user);
           }

           // удалить роль
           roleRepository.deleteByTitle(roleTitle);

           // фиксация транзакции
           transactionManager.commit(status);
       }
       catch (DataAccessException ex)
       {
           // откатить транзакцию в случае ошибки
           transactionManager.rollback(status);
           throw ex;
       }
   }
}
```

Листинг 21 - пример ручного управления транзакциями

ПРИЛОЖЕНИЕ Е
Примеры автотестов

```
package ru.naumen.NauJava;

import java.util.UUID;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

import ru.naumen.NauJava.entity.User;
import ru.naumen.NauJava.repository.UserRepository;

@SpringBootTest
class UserTest
{
   private final UserRepository userRepository;

   @Autowired
   UserTest(UserRepository userRepository)
   {
       this.userRepository = userRepository;
   }

   /**
    * Тестирование поиск пользователя по его имени
    */
   @Test
   void testFindUserByName()
   {
       // генерация имени пользователя
       String userName = UUID.randomUUID().toString();

       // создание пользователя
       User user = new User();
       user.setName(userName);
       userRepository.save(user);

       // поиск пользователя по имени
       User foundUser = userRepository.findByName(userName).getFirst();

       // проверки
       Assertions.assertNotNull(foundUser);
       Assertions.assertEquals(user.getId(), foundUser.getId());
       Assertions.assertEquals(userName, foundUser.getName());
   }
}
```

Листинг 22 - пример автотеста для проверки методов репозитория

```java
package ru.naumen.NauJava;

import java.util.Optional;
import java.util.UUID;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

import ru.naumen.NauJava.entity.Role;
import ru.naumen.NauJava.entity.User;
import ru.naumen.NauJava.repository.RoleRepository;
import ru.naumen.NauJava.repository.UserRepository;
import ru.naumen.NauJava.service.RoleService;

@SpringBootTest
class RoleTest
{
   private final RoleService roleService;
   private final UserRepository userRepository;
   private final RoleRepository roleRepository;

   @Autowired
   public RoleTest(RoleService roleService,
               UserRepository userRepository,
               RoleRepository roleRepository)
   {
       this.roleService = roleService;
       this.userRepository = userRepository;
       this.roleRepository = roleRepository;
   }

   /**
    * Тестирование удаления роли вместе с пользователями
    */
   @Test
   void testDeleteRoleInTx()
   {
       // создание роли
       Role adminRole = new Role();
       adminRole.setTitle(UUID.randomUUID().toString());
       roleRepository.save(adminRole);

       // создание пользователя 1
       User user1 = new User();
       user1.setName(UUID.randomUUID().toString());
       user1.setRole(adminRole);
       userRepository.save(user1);

       // создание пользователя 2
       User user2 = new User();
       user2.setName(UUID.randomUUID().toString());
       user2.setRole(adminRole);
       userRepository.save(user2);

       // удаление роли
       roleService.deleteRole(adminRole.getTitle());

       // проверка отсутствия роли в БД
       Optional<Role> foundRole = roleRepository.findById(adminRole.getId());
       Assertions.assertTrue(foundRole.isEmpty());

       // проверка отсутствия пользователя 1 в БД
       Optional<User> foundUser1 = userRepository.findById(user1.getId());
       Assertions.assertTrue(foundUser1.isEmpty());

       // проверка отсутствия пользователя 2 в БД
       Optional<User> foundUser2 = userRepository.findById(user2.getId());
       Assertions.assertTrue(foundUser2.isEmpty());
   }
}
```

Листинг 23 - пример автотеста для проверки транзакционного метода